



Universidad Nacional Experimental de Guayana

Coordinación General de Pregrado

Proyecto de Carrera: Ingeniería en Informática

Unidad Curricular: Lenguajes y Compiladores

Sección: 1

Tema 5 - Análisis Sintáctico

Pregunta 1, 2 y 3

Profesor:

Felix Marquez

Alumnos:

Iván Hernández - C.I: 30.529.821

Miguel Gómez - C.I: 29.862.177

Miguel Barrios - C.I: 28.619.898

Daniel Valenzuela - C.I: 28.619.852

Puerto Ordaz, 19 de julio de 2026.

Introducción

El desarrollo de compiladores e intérpretes modernos constituye uno de los pilares fundamentales de la informática, permitiendo traducir las abstracciones de los lenguajes de alto nivel a instrucciones comprensibles por las máquinas. Este proceso se ejecuta a través de fases estrictamente encadenadas. Tras completarse el análisis léxico (Lexer), cuya función es agrupar los caracteres del código fuente en unidades con significado lógico llamadas tokens, el flujo avanza hacia la segunda gran etapa: el Análisis Sintáctico (Parser).

El propósito principal del analizador sintáctico es evaluar si la secuencia lineal de tokens enviada por el lexer cumple rigurosamente con las reglas y producciones estructurales definidas en una Gramática Libre de Contexto (GLC). Para lograr esto de forma eficiente, el parser no sólo valida la legalidad de las "oraciones" del código, sino que transforma esa lista plana de componentes en una estructura jerárquica de gran valor lógico: el Árbol de Sintaxis Abstracta (AST), el cual sirve de base para las subsiguientes validaciones semánticas y optimizaciones.

Árbol de Sintaxis Abstracta (AST)

Definición y Características

Un Árbol de Sintaxis Abstracta (AST) es una representación jerárquica y en forma de grafo (árbol) de la estructura sintáctica abstracta del código fuente escrito en un lenguaje de programación. El AST es "abstracto" porque elimina detalles puramente sintácticos y redundantes del código, como los paréntesis explicativos, los puntos y comas de fin de instrucción, o las palabras clave que solo sirven como delimitadores en la gramática.

Sus características principales son:

- **Jerarquía Lógica:** Los nodos internos del árbol representan operadores o estructuras de control (como sumas, asignaciones o bucles), mientras que los nodos hojas representan los operandos o valores literales (variables y constantes).
- **Simplicidad:** Facilita las fases posteriores del compilador (análisis semántico y generación de código) al presentar la lógica del programa limpia de ruido gramatical.
- **Independencia del Formato:** No importa cuántos espacios en blanco o líneas adicionales use el programador, el AST generado será estructuralmente idéntico.
- **Soporte para el Análisis Semántico:** El AST le deja el trabajo en bandeja de plata al compilador para hacer chequeos lógicos. En vez de leer texto plano, el compilador mira el árbol para verificar cosas como: no sumar una palabra con un número (error de tipo), o que no se está usando una variable que ni siquiera se ha creado todavía.
- **Facilidad de Transformación y Optimización:** El compilador puede reordenar o simplificar las ramas para que el programa final sea más rápido y eficiente. Por ejemplo, si en el código aparece $x = 5 * 2$, el compilador calcula el **10** directamente en el árbol para ahorrarle trabajo a la computadora (plegado de constantes), o borra por completo las líneas de código que detecte que nunca se van a llegar a ejecutar (código muerto).

- **Desacoplamiento del Backend:** El árbol funciona como un "molde intermedio" universal. Gracias a esto, se puede usar el mismo AST para generar código que corra en una PC normal (arquitectura x86) o en un teléfono inteligente (arquitectura ARM), sin tener que volver a analizar la sintaxis de todo el archivo desde cero.

Ejemplos de Entrada y Representación

Para entender cómo se estructuran, analicemos dos ejemplos clásicos de entrada:

Ejemplo A: Operación Aritmética

- **Entrada:** $3 + 5 * 2$
- **Estructura del AST:** La multiplicación posee mayor prioridad, por lo que se agrupa en un nivel inferior para ejecutarse primero.

```

      +
     / \
    3   *
       / \
      5  2

```

Ejemplo B: Asignación de Variable

- **Entrada:** $x = 10$
- **Estructura del AST:** El operador de asignación ($=$) actúa como la raíz, teniendo a la variable a la izquierda y el valor asignado a la derecha.

```

    =
   / \
  x  10

```

Implementación de Ejemplo (Python)

Este es el código en Python puro que implementa el nodo base y construye el árbol para la expresión $3 + 5 * 2$:

```
class NodoAST:

    def __init__(self, tipo_token, valor, izquierdo=None, derecho=None):

        self.tipo_token = tipo_token # OPERADOR, NUMERO, VARIABLE

        self.valor = valor           # Lexema real (+, *, 3, etc.)

        self.izquierdo = izquierdo  # Hijo izquierdo

        self.derecho = derecho       # Hijo derecho


    def __str__(self, nivel=0):

        # Genera la visualización jerárquica del árbol en consola

        resultado = "    " * nivel + f"└── [{self.tipo_token}: {self.valor}]\n"

        if self.izquierdo:

            resultado += self.izquierdo.__str__(nivel + 1)

        if self.derecho:

            resultado += self.derecho.__str__(nivel + 1)

        return resultado


if __name__ == "__main__":

    print("--- Representación Jerárquica del AST para: 3 + 5 * 2 ---")

    # Construcción de las hojas (Operandos)

    num3 = NodoAST("NUMERO", "3")

    num5 = NodoAST("NUMERO", "5")
```

```
num2 = NodoAST("NUMERO", "2")
```

```
# Subárbol de mayor prioridad (Multiplicación)
```

```
nodo_mult = NodoAST("OPERADOR", "*", num5, num2)
```

```
# Raíz del árbol (Suma)
```

```
raiz_ast = NodoAST("OPERADOR", "+", num3, nodo_mult)
```

```
print(raiz_ast)
```

Comparación entre los Análisis LL y LR

Definición y Criterios Comparativos

Los analizadores sintácticos se dividen fundamentalmente en dos estrategias de recorrido y construcción según la gramática formal utilizada:

Criterio	Análisis LL (Left-to-right, Leftmost derivation)	Análisis LR (Left-to-right, Rightmost derivation)
Dirección y Derivación	Lee de izquierda a derecha y construye una derivación por la izquierda.	Lee de izquierda a derecha y construye una derivación inversa por la derecha.
Enfoque	Descendente: Comienza desde el símbolo inicial de la gramática e intenta expandirlo hacia abajo hasta emparejar los tokens de entrada.	Ascendente: Comienza desde los tokens de entrada (las hojas) y realiza reducciones sucesivas hacia arriba hasta alcanzar el símbolo inicial.
Complejidad	Baja: Son intuitivos de entender y muy sencillos de implementar manualmente a través de funciones recursivas.	Alta: Requieren la construcción de complejas tablas de transiciones de estados de gran tamaño matemático.
Flexibilidad	Restictivo: No admiten ambigüedades ni gramáticas	General: Maneja gramáticas mucho más amplias,

	con recursión por la izquierda (causan bucles infinitos).	complejas y expresiones anidadas con total soltura.
--	--	--

Evidencia de Uso Actual

- **Uso de LL:** Los compiladores modernos de lenguajes como **Python** y **Java** han migrado o implementado analizadores descendentes basados en variantes avanzadas de LL (como los parsers recursivos descendentes por PEG o LL(k)), debido a que facilitan enormemente el control manual y la personalización de mensajes de error contextuales.
- **Uso de LR:** Herramientas tradicionales del sistema de compilación de **GCC** para lenguajes como C/C++ han dependido históricamente de motores analíticos basados en LALR(1) (un subconjunto eficiente de LR) generados de manera automatizada.

Implementación Comparativa para un mismo Lenguaje SL

Consideremos un lenguaje mínimo SL cuya única regla válida es una secuencia de dos números enteros separados por un operador de suma (ej: $INT + INT$).

Caso LL: Parser Recursivo Descendente Manual (Python)

Este algoritmo busca proactivamente el primer componente esperado y desciende secuencialmente:

```
class ParserLL1Manual:
    def __init__(self, lista_tokens):
        self.tokens = lista_tokens
        self.indice = 0
```



```

def obtener_token_actual(self):
    if self.indice < len(self.tokens):
        return self.tokens[self.indice]
    return "EOF"

def emparejar(self, token_esperado):
    token_actual = self.obtener_token_actual()
    if token_actual == token_esperado:
        print(f" [OK] Token emparejado correctamente: {token_esperado}")
        self.indice += 1
    else:
        print(f" [ERROR SINTÁCTICO] Se esperaba '{token_esperado}' pero se encontró
'{token_actual}'")
        raise SyntaxError("Error sintáctico en LL(1)")

def analizar(self):
    print("[INFO] Iniciando análisis de la regla: NUMERO + NUMERO")
    self.emparejar("TK_INT")
    self.emparejar("TK_PLUS")
    self.emparejar("TK_INT")

    if self.obtener_token_actual() == "EOF":
        print("\n>> ¡ANÁLISIS EXITOSO! La estructura sintáctica es 100% válida. <<")
    else:
        raise SyntaxError("Tokens residuales al final del archivo.")

```

```

if __name__ == "__main__":
    print("=== SCRIPT 2: MICRO-PARSER RECURSIVO DESCENDENTE LL(1) ===\n")

    # Prueba 1: Éxito
    tokens_validos = ["TK_INT", "TK_PLUS", "TK_INT"]
    print(f"--- Ejecutando Prueba 1 con tokens válidos: {tokens_validos} ---\n")
    parser1 = ParserLL1Manual(tokens_validos)
    try:
        parser1.analizar()
    except SyntaxError:
        pass

    print("\n" + "="*65 + "\n")

    # Prueba 2: Error
    tokens_invalidos = ["TK_INT", "TK_PLUS"]
    print(f"--- Ejecutando Prueba 2 con tokens inválidos: {tokens_invalidos} ---\n")
    parser2 = ParserLL1Manual(tokens_invalidos)
    try:
        parser2.analizar()
    except SyntaxError:
        print("\n>> El Parser contuvo el error de manera controlada como se esperaba. <<")

```

Caso LR: Simulación Basada en Tabla de Desplazamiento/Reducción (Python)

El enfoque descendente maneja una pila de estados y tokens, realizando acciones de *Shift* (desplazar) o *Reduce* (reducir a la regla base):

```
class ParserLRSimulado:
```

```
    def __init__(self, lista_tokens):
```

```
        self.tokens = lista_tokens + ["$"]
```

```
        self.pila_estados = [0]
```

```
        self.pila_simbolos = []
```

```
    def analizar(self):
```

```
        print("[INFO] Iniciando análisis LR(1) ascendente (Shift / Reduce)")
```

```
        idx = 0
```

```
        while True:
```

```
            token = self.tokens[idx]
```

```
            estado_actual = self.pila_estados[-1]
```

```
            print(f" Pila Estados: {self.pila_estados} | Símbolos: {self.pila_simbolos} | Token actual: '{token}'")
```

```
            if estado_actual == 0 and token == "TK_INT":
```

```
                self.pila_simbolos.append(token)
```

```
                self.pila_estados.append(1)
```

```
                print(" --> [SHIFT] Desplazando 'TK_INT' al Estado 1")
```

```
                idx += 1
```

```
elif estado_actual == 1 and token == "TK_PLUS":
```

```
    self.pila_simbolos.append(token)
```

```
    self.pila_estados.append(2)
```

```
    print(" --> [SHIFT] Desplazando 'TK_PLUS' al Estado 2")
```

```
    idx += 1
```

```
elif estado_actual == 2 and token == "TK_INT":
```

```
    self.pila_simbolos.append(token)
```

```
    self.pila_estados.append(3)
```

```
    print(" --> [SHIFT] Desplazando 'TK_INT' al Estado 3")
```

```
    idx += 1
```

```
elif estado_actual == 3 and token == "$":
```

```
    print(" --> [REDUCE] Reduciendo [TK_INT TK_PLUS TK_INT] ->
```

```
<EXPRESION_SUMA>")
```

```
    self.pila_simbolos = ["<EXPRESION_SUMA>"]
```

```
    print("\n>> ¡ANÁLISIS EXITOSO! La estructura sintáctica es 100% válida. <<")
```

```
    break
```

```
else:
```

```
    print(f" [ERROR SINTÁCTICO] Se esperaba token válido pero se encontró '{token}'")
```

```
    raise SyntaxError("Error sintáctico en LR(1)")
```

```
if __name__ == "__main__":
```

```
    print("=== SCRIPT 3: PARSER ASCENDENTE LR(1) (SHIFT / REDUCE) ===\n")
```

```
# Prueba 1: Éxito

tokens_validos = ["TK_INT", "TK_PLUS", "TK_INT"]

print(f"--- Ejecutando Prueba 1 con tokens válidos: {tokens_validos} ---\n")

parser1 = ParserLRSimulado(tokens_validos)

try:

    parser1.analizar()

except SyntaxError:

    pass


print("\n" + "="*65 + "\n")


# Prueba 2: Error

tokens_invalidos = ["TK_INT", "TK_PLUS"]

print(f"--- Ejecutando Prueba 2 con tokens inválidos: {tokens_invalidos} ---\n")

parser2 = ParserLRSimulado(tokens_invalidos)

try:

    parser2.analizar()

except SyntaxError:

    print("\n>> El Parser contuvo el error de manera controlada como se esperaba. <<")
```

Generadores de Analizadores Sintácticos y Gestión de Errores

Los Generadores de Analizadores Sintácticos (también denominados metacompiladores o *compiler-compilers*) son frameworks de software especializados que toman como archivo de entrada la especificación formal de una gramática (expresada en Metalenguajes como la Notación Backus-Naur - BNF) y generan de manera automatizada el código fuente de un lexer y un parser funcional.

Principales Herramientas Actuales

- **Bison:** Un generador clásico del proyecto GNU compatible con reglas LALR(1) y deterministas, utilizado comúnmente junto a Flex en entornos basados en C/C++.
- **ANTLR (ANother Tool for Language Recognition):** Un potente metacompilador moderno que genera parsers de tipo ALL(*) (un enfoque descendente avanzado) y cuenta con la versatilidad de exportar el código resultante hacia múltiples lenguajes de programación de destino, tales como Java, Python, C++, y C#.

Estrategias de Gestión y Recuperación de Errores

Cuando la cadena de tokens rompe las reglas de la gramática formal, un parser robusto no debe colapsar inmediatamente de manera destructiva; debe aplicar técnicas de contención para seguir evaluando el resto del archivo de texto en busca de más fallos. Las dos técnicas principales provistas por estas herramientas son:

1. **Modo Pánico (Panic Mode Recovery):** Al detectarse la discrepancia sintáctica, el parser descarta de forma consecutiva los tokens entrantes uno por uno hasta encontrarse con un delimitador seguro previamente configurado, denominado **token de sincronización** (tales como un punto y coma `;`, una llave de cierre `}`, o un salto de línea `\n`). Una vez localizado el token de sincronización, el parser limpia su pila de llamadas,

asume un estado consistente y reanuda el análisis normal a partir de la siguiente línea o instrucción.

2. **Inserción de Tokens Faltantes (Token Insertion / Deletion):** Consiste en una corrección lógica en caliente. Si el parser detecta que la sintaxis es inválida pero se solucionaría de manera matemática insertando un token predecible faltante (por ejemplo, insertar el paréntesis de cierre `)` en una estructura condicional `if (x == 5)`, el motor sintáctico inyecta de forma virtual dicho token en el flujo, registra la advertencia detallada en el reporte de errores y prosigue con la construcción limpia del Árbol de Sintaxis Abstracta.
3. **Recuperación a Nivel de Frase (Phrase-level Recovery):** Consiste en hacer una "microcirugía" local en los tokens justo donde ocurrió el fallo. En lugar de ignorar texto o saltar líneas, el parser realiza un reemplazo inmediato para intentar salvar la instrucción. Por ejemplo, si por error hay una coma donde iba un punto y coma (como `int x = 5, )`, el parser cambia localmente esa `,` por un `;` en su memoria y continúa analizando el resto del archivo sin detenerse.
4. **Producciones de Error (Error Productions):** Esta técnica consiste en "predecir" los errores más comunes de los humanos y agregarlos a propósito como reglas válidas dentro de la gramática del compilador. Si el diseñador del lenguaje sabe que los estudiantes siempre olvidan escribir la palabra clave `then` después de un `if`, añade una regla extra en la gramática que diga: *"Si el usuario escribe el `if` sin el `then`, acéptalo igual para que el programa no colapse, pero lánzale un reporte que diga: '¡Hey, olvidaste poner el then!'"*.

Conclusión

El análisis sintáctico es el núcleo estructurador del proceso de compilación, actuando como el puente definitivo entre el texto plano y la lógica ejecutable de un programa. A través del estudio del Árbol de Sintaxis Abstracta (AST), se evidencia cómo este ayuda a purificar el código fuente, abstrayendo elementos redundantes, simplificando estructuras complejas y permitiendo continuar con la correcta compilación del código.

De la evaluación comparativa entre un enfoque descendente (LL) y uno ascendente (LR) se obtuvo que los parsers LL destacan por su baja complejidad y su facilidad de implementación manual mediante funciones recursivas (ampliamente valorados en compiladores modernos por la precisión con la que reportan errores al usuario), mientras que los motores LR ofrecen una flexibilidad gramatical superior, siendo capaces de resolver estructuras y expresiones anidadas altamente complejas a costa de una mayor densidad matemática en sus tablas de estados.

Por último, el análisis de las herramientas de metacompilación como ANTLR y Bison resalta la importancia de la robustez industrial en el software actual. Un parser no puede limitarse a ser un juez punitivo que colapse ante el primer fallo sintáctico del desarrollador; la implementación de técnicas de ingeniería como el modo pánico y la inyección virtual de tokens faltantes otorga a los entornos de desarrollo de la tolerancia a fallos necesaria para continuar el procesamiento secuencial del archivo. En última instancia, comprender el funcionamiento de estas estructuras son las que otorgan el criterio técnico para diseñar lenguajes robustos, optimizar software y auditar con propiedad las herramientas probabilísticas de asistencia actuales.

Referencias bibliográficas

Aho, A. V., Lam, M. S., Sethi, R., y Ullman, J. D. (2008). *Compiladores: Principios, técnicas y herramientas* (2da ed.). Pearson Educación.

Márquez, F. (2026). *Tema 5: Análisis Sintáctico* [Material de lectura en PDF]. Coordinación de Ingeniería en Informática, Universidad Nacional Experimental de Guayana (UNEG).

Parr, T. (2013). *The Definitive ANTLR 4 Reference* (2da ed.). Pragmatic Bookshelf.

Python Software Foundation. (2026). *difflib — Helpers for computing deltas* [Documentación oficial de Python]. <https://docs.python.org/3/library/difflib.html>.